Taller Web

1. Objetivo

Diseñar una página web donde se permita añadir tareas y usar el LocalStorage para aprender el flujo de los datos en una interfaz, el almacenamiento local y los formatos Semi-estructurados.

2. Estructura del proyecto

```
Taller-web/
|-- index.html (estructura del formulario)
|-- estilos.css (hoja de estilos externa)
`-- aplicacion.js (lógica en JavaScript)
```

3. Repositorio Git

https://github.com/matsuro262/aplicaciones_cliente_web.git

El proyecto se sube como una subcarpeta (taller-web) dentro del repositorio general de la materia, junto a los demás trabajos.

4. Código HTML

```html
<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Taller: Gestor de Tareas Semiestructurado</title>
    <link rel="stylesheet" href="estilos.css" />
  </head>
  <body>
    <div class="contenedor">
      <h1>Gestor de Tareas</h1>

      <form id="formulario-tareas">
        <input
          type="text"
          id="titulo-tarea"
          placeholder="Título de la tarea"
          required
        />
        <textarea
          id="descripcion-tarea"
          placeholder="Descripción de la tarea"
          required
        ></textarea>
        <button type="submit">Agregar Tarea</button>
      </form>
```

```
      <div class="acciones-datos">
        <button id="btn-exportar-json" class="btn-secundario">
          Exportar a JSON
        </button>
        <button id="btn-exportar-xml" class="btn-secundario">
          Exportar a XML
        </button>
      </div>
      <h2>Mis Tareas Activas</h2>
      <ul id="lista-tareas"></ul>
    </div>
    <script src="aplicacion.js"></script>
  </body>
</html>
```

5. Código CSS

La hoja de estilos usa una paleta en tonos azules, celestes y blancos.

- Fondo de página con un gris suave.
- Campos de entrada con borde suave que se resalta en negro al dar clic.
- Botón de “Agregar tarea” en azul, y botones de exportar tanto a Json como XML en un celeste claro

```
:root {
    --color-primario: #4f46e5;
    --color-secundario: #0ea5e9;
    --color-fondo: #f8fafc;
    --color-tarjeta: #ffffff;
    --color-texto: #0f172a;
}

body {
    font-family: 'Segoe UI', sans-serif;
    background-color: var(--color-fondo);
    color: var(--color-texto);
    margin: 0;
    padding: 20px;
}

.contenedor {
    max-width: 600px;
    margin: 0 auto;
    background: var(--color-tarjeta);
    padding: 30px;
    border-radius: 12px;
```

```
    box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
}

h1,
h2 {
    text-align: center;
    color: var(--color-primario);
}

form {
    display: flex;
    flex-direction: column;
    gap: 12px;
    margin-bottom: 20px;
}

input,
textarea {
    padding: 10px;
    border: 1px solid #cbd5e1;
    border-radius: 6px;
    font-size: 16px;
}

button {
    background-color: var(--color-primario);
    color: white;
    border: none;
    padding: 12px;
    border-radius: 6px;
    cursor: pointer;
    font-size: 16px;
    font-weight: bold;
    transition: background 0.3s;
}

button:hover {
    background-color: #4338ca;
}

.acciones-datos {
    display: flex;
    gap: 10px;
    margin-bottom: 20px;
}
```

```
.btn-secundario {
    background-color: var(--color-secundario);
    flex: 1;
}

.btn-secundario:hover {
    background-color: #0284c7;
}

ul {
    list-style: none;
    padding: 0;
}

.elemento-tarea {
    background: #f1f5f9;
    padding: 15px;
    border-left: 5px solid var(--color-primario);
    margin-bottom: 10px;
    border-radius: 0 6px 6px 0;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.elemento-tarea h3 {
    margin: 0 0 5px 0;
}

.elemento-tarea p {
    margin: 0;
    color: #475569;
    font-size: 14px;
}

.btn-eliminar {
    background-color: #ef4444;
    padding: 6px 12px;
    font-size: 14px;
}

.btn-eliminar:hover {
    background-color: #dc2626;
}
```

6. Código JavaScript (lógica de cálculo)

El script sigue los siguientes pasos:

1. Referencia al modelo de objetos del documento (el DOM)
2. Carga los datos previos del LocalStorage o inicia en vacío
3. Pinta las tareas en el html
4. Guarda datos nativos en LocalStorage convirtiendo el texto a Json
5. Captura el envío del formulario
6. Dependiendo de lo que elija el usuario, le permitirá exportarlo a JSON o a XML.

```
// 1. Referencias al Modelo de Objetos del Documento (DOM)
const formularioTareas = document.getElementById('formulario-tareas');
const tituloTarea = document.getElementById('titulo-tarea');
const descripcionTarea = document.getElementById('descripcion-tarea');
const listaTareas = document.getElementById('lista-tareas');
const btnExportarJson = document.getElementById('btn-exportar-json');
const btnExportarXml = document.getElementById('btn-exportar-xml');
// 2. Estado de la Aplicación (Cargar datos previos de LocalStorage o iniciar
vacío)
let coleccionTareas = JSON.parse(localStorage.getItem('tareas Guardadas')) || [];
// 3. Función para pintar las tareas en el HTML
function redibujarInterfaz() {
    listaTareas.innerHTML = ''; // Vaciar contenedor antes de actualizar

    coleccionTareas.forEach((tarea, indice) => {
        const elementoLista = document.createElement('li');
        elementoLista.className = 'elemento-tarea';

        elementoLista.innerHTML = `
 <div>
 <h3>${tarea.titulo}</h3>
 <p>${tarea.descripcion}</p>
 <small style="color: #94a3b8;">Código: ${tarea.codigo} | Registro:
${tarea.fecha}</small>
 </div>
 <button class="btn-eliminar"
onclick="removerTarea(${indice})">Eliminar</button>
 `;
        listaTareas.appendChild(elementoLista);
    });
}
```

```
// 4. Guardar datos nativos en LocalStorage convirtiéndolos a texto JSON
function actualizarAlmacenamientoLocal() {
    localStorage.setItem('tareasGuardadas', JSON.stringify(coleccionTareas));
}
// 5. Captura del evento de envío del formulario
formularioTareas.addEventListener('submit', (evento) => {
    evento.preventDefault(); // Evita que la página se recargue
    // Creamos la estructura del objeto con claves descriptivas en español
    const nuevaTarea = {
        codigo: Date.now().toString(),
        titulo: tituloTarea.value,
        descripcion: descripcionTarea.value,
        fecha: new Date().toLocaleDateString()
    };
    coleccionTareas.push(nuevaTarea);
    actualizarAlmacenamientoLocal();
    redibujarInterfaz();

    formularioTareas.reset(); // Limpiar campos de texto
});
// 6. Eliminar elemento del arreglo por su índice
window.removerTarea = function (indice) {
    coleccionTareas.splice(indice, 1);
    actualizarAlmacenamientoLocal();
    redibujarInterfaz();
};
// ========================================================
// PROCESAMIENTO DE FORMATOS SEMIESTRUCTURADOS (XML y JSON)
// ========================================================
// Exportación nativa a formato JSON
btnExportarJson.addEventListener('click', () => {
    if (coleccionTareas.length === 0) return alert('No existen tareas para
exportar.');

    // Serialización: de objeto JavaScript en memoria a cadena estructurada JSON
    const textoJson = JSON.stringify(coleccionTareas, null, 2);

    console.log("--- FLUJO DE DATOS: JSON GENERADO ---");
    console.log(textoJson);

    generarDescarga(textoJson, 'tareas_academicas.json', 'application/json');
});
// Exportación estructurada a formato XML mediante marcado manual
btnExportarXml.addEventListener('click', () => {
```

```
    if (coleccionTareas.length === 0) return alert('No existen tareas para
exportar.');
    // Construcción de la cabecera e inicio del nodo raíz
    let textoXml = `<?xml version="1.0" encoding="UTF-8"?>\n<tareas>\n`;

    // Iteración para anidar nodos secundarios estructurados
    coleccionTareas.forEach(tarea => {
        textoXml += `  <tarea codigo="${tarea.codigo}">\n`;
        textoXml += `  <titulo>${sanitizarTextoXml(tarea.titulo)}</titulo>\n`;
        textoXml += `
<descripcion>${sanitizarTextoXml(tarea.descripcion)}</descripcion>\n`;
        textoXml += `  <fecha>${tarea.fecha}</fecha>\n`;
        textoXml += `  </tarea>\n`;
    });

    textoXml += `</tareas>`; // Cierre del nodo raíz

    console.log("--- FLUJO DE DATOS: XML GENERADO ---");
    console.log(textoXml);

    generarDescarga(textoXml, 'tareas_academicas.xml', 'application/xml');
});
// Función utilitaria para disparar la descarga de archivos en el cliente
function generarDescarga(contenidoTexto, nombreArchivo, tipoMime) {
    const bloqueDatos = new Blob([contenidoTexto], { type: tipoMime });
    const urlDescarga = URL.createObjectURL(bloqueDatos);
    const enlaceDescarga = document.createElement('a');

    enlaceDescarga.href = urlDescarga;
    enlaceDescarga.download = nombreArchivo;
    enlaceDescarga.click();

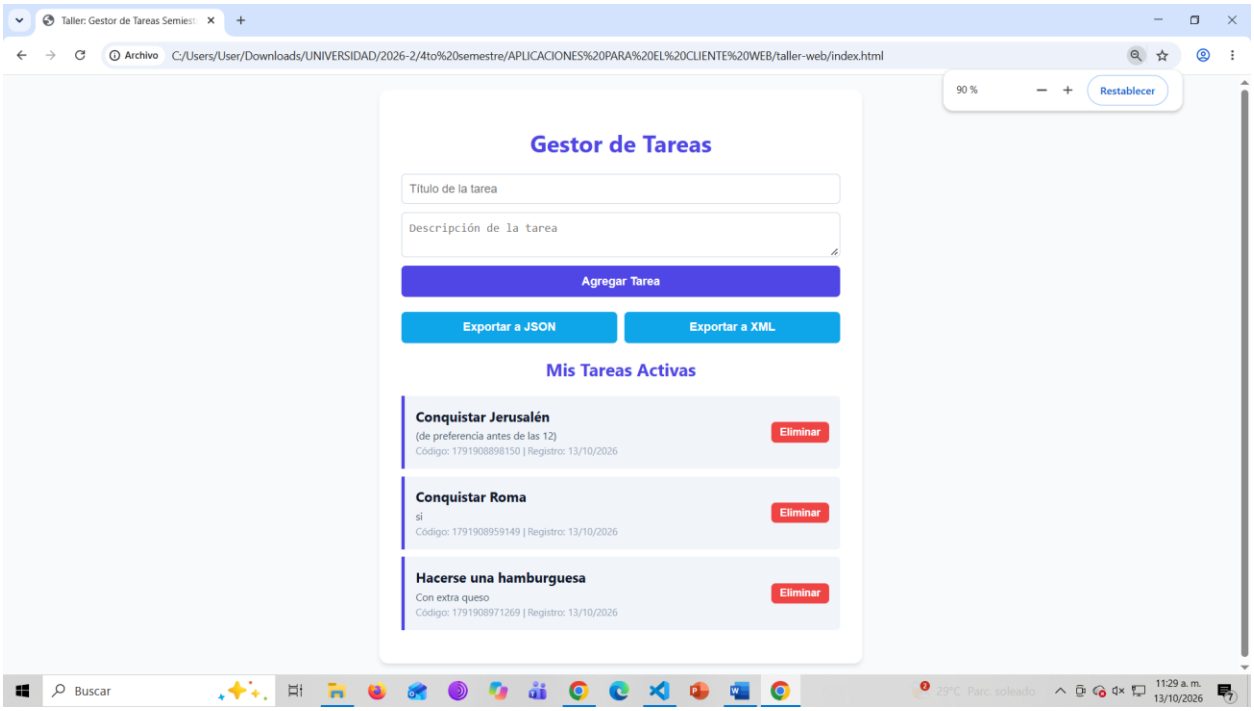
    URL.revokeObjectURL(urlDescarga); // Liberar memoria del navegador
}
// Sanitización para prevenir errores de parsing si el usuario ingresa caracteres
reservados de XML
function sanitizarTextoXml(textoInseguro) {
    return textoInseguro.replace(/[<>&'"]/g, (caracter) => {
        switch (caracter) {
            case '<': return '&lt;';
            case '>': return '&gt;';
            case '&': return '&amp;';
            case '\'': return '&apos;';
            case '"': return '&quot;';
        }
```

```
    });
}
// 7. Renderizado inicial al cargar la ventana del navegador por primera vez
redibujarInterfaz();
```

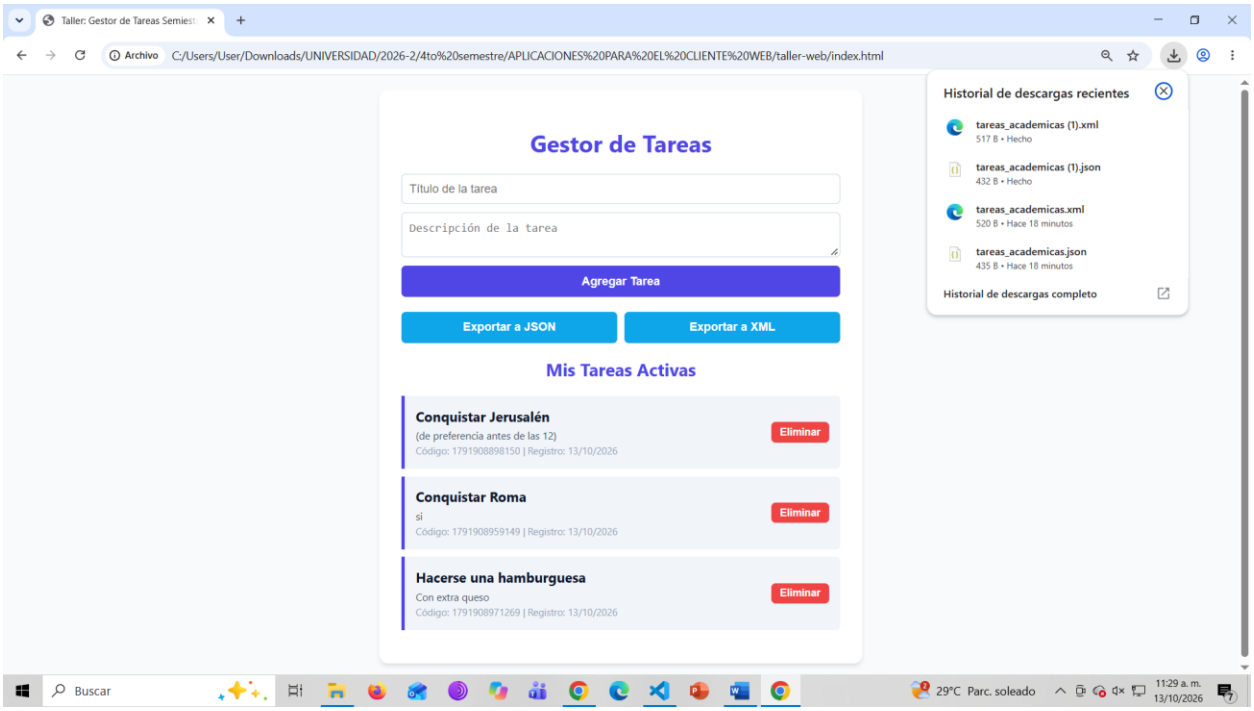
7. Ejecución con el formulario vacío



8. Ejecución – Ingresar 3 tareas



9. Ejecución – Exportar a JSON y XML



Nota: Tuve que crear nuevamente tres tareas porque cerré la ventana donde ya tenía las otras.

10. Comprobación del LocalStorage

